

Huttle AI Codebase Audit — Grok 4.3 Migration, Billing, Tier Limits, Dashboard & Migration State

Full-repo read-only review answering the Section A–E questions on Grok cost controls, Stripe/subscription write paths, Plan Builder tier gating, dashboard cache behavior, and migration drift. Zero code changes made.

PRODUCT	Huttle AI
DATE	25 August 2026, 22:07 (America/New_York)
BRANCH CHECKED OUT	main
HEAD	a1f3674 (2026-08-25 21:56 -04:00)
METHOD	Static read-only review (grep/read/git show); no checkout switches, no writes
CLASSIFICATION	Internal / staff engineer

PREMISE CORRECTION

Branch context is wrong

grok-4.3-migration is **not** an unmerged preview branch. It was merged into main via PR #96 (4f036d1). main is strictly ahead by 8 commits — there is no separate branch state to diff.

VERIFIED

Grok cost controls are sound

Every Grok call site is forced through a single getGrokParams(featureKey) helper. Reasoning effort is never omitted, and the client-supplied model field is logged but never used — no allowlist gap exists because there is nothing to allowlist.

GAP FOUND

tier_feature_limits is dead weight

Zero application code and zero migrations reference the `tier_feature_limits` table. The Aug 25 `essentials_trial / pro_trial` rows added to it have no functional effect on enforcement.

No application source was edited. No files were modified prior to this report. All findings are cited to exact file paths and line numbers as they existed at HEAD `a1f3674` on `main`.

Contents

- Branch-context correction
- A1—Reasoning effort coverage
- A2—Model strings
- A3—Max tokens per feature
- A4—Client-supplied model (security)
- A5—Full Post Builder cost & routing
- B1— `subscriptions` writers
- B2—Stripe webhook handler
- B3—Checkout session creation
- B4—Trial reminder cron
- B5—Hardcoded price IDs
- C1—14-day Plan Builder gating
- C2—Credit-checking module
- D1—Dashboard cache read query
- D2—Perplexity timeout / retry
- D3— `signOut()` call sites
- E1—Ten most recent migrations
- E2—Migration references vs. Aug 25 prod changes
- Summary of corrections

Branch-context correction — read this first

The audit brief states that `grok-4.3-migration` is a preview branch containing unmerged work. This is factually incorrect:

```
git merge-base main grok-4.3-migration → a8b37b7 (tip of grok-4.3-migration)
git log main..grok-4.3-migration        → empty (zero commits unique to migration)
git log grok-4.3-migration..main        → 8 commits, starting with the merge commit itself
```

```
alf3674 perf(niche-intel): give deep research its own runtime budget
92b785c fix(full-post): drop degraded-mode notices that fired on every run
c22e745 test(qa): add verification harness for the 2026-08-25 audit fixes
4c76ace fix(subscription): log the real reason a database read failed
13f7e9e fix(ignite): stop discarding slow n8n briefs and rendering raw null
3b9423b fix(full-post): stop duplicating the hook and CTA in the saved post
75d4437 fix(billing): resolve start and renewal dates from the subscription item
6bc93c8 fix(niche-intel): surface run failures instead of reverting to empty state
4f036d1 Merge pull request #96 from huttleai/grok-4.3-migration
a8b37b7 feat(ai): migrate all Grok call sites to grok-4.3 with per-feature reasoning effort
```

`main` = `grok-4.3-migration` + 8 additional post-merge commits. There is no separate preview state to diff; every finding below applies identically to both labels since they share the same lineage. All answers are given once, against `main`.

Section A — Grok 4.3 migration

A1 · HIGHEST COST PRIORITY

Reasoning effort coverage

Reasoning effort is **never omitted**. Every call site is forced through `getGrokParams(featureKey)` in `src/config/grokConfig.js:23-52,61-81`, which returns `{ model, reasoning_effort }` from a hardcoded map or throws in dev / logs+defaults to `'none'` in prod for an unrecognized key. No call site skips this helper.

REASONING EFFORT	FEATURE KEYS
'none' (17)	caption, hookBuilder, hashtag, cta, contentQualityScorer, visualBrainstorm, dashboardWidget, optimizeTimes, performancePrediction, voiceTranscriptPolish, captionVariations, humanizerScore , trendIdeas, improveContent, visualIdeas, autoImprovePhrase, perplexityGrokFallback
'low' (8)	fullPostBuilder , contentRemix, igniteEngine, contentRepurposer, contentPlan, platformRemixes, audienceInsights, nicheIntel

Comment at `src/config/grokConfig.js:21`: *“This pass uses only 'none' and 'low' — do not add 'medium'/'high' without approval.”* Server-side, `api/ai/grok.js:377,403-405` re-validates the echoed `reasoning_effort` against `{none,low,medium,high}` and defaults to `'none'` on anything else.

No cost risk found. No Grok call site bypasses `getGrokParams`.

A2

Model strings

Exactly one Grok model string exists in the entire repository:

```
15:15:src/config/grokConfig.js
export const GROK_MODEL = 'grok-4.3';
```

`grok-4-1-fast-reasoning` and every other legacy Grok 2/3/4 slug — **NOT FOUND** anywhere in `src/`, `api/`, or `scripts/`. All nine call sites (`api/ai/grok.js`, `src/services/grokAPI.js`, `src/services/perplexityAPI.js`, `src/services/optimizeTimesAPI.js`, `src/services/dashboardCacheService.js` x3, `src/pages/IgnoreEngine.jsx`, `src/pages/ContentRepurposer.jsx`) resolve the model exclusively via `getGrokParams(featureKey)`.

A4 · SECURITY

Client-supplied model

The premise is half right and the important half is wrong: the client-supplied `model` field is received and logged, but **never used**.

```
402:405:api/ai/grok.js
// Model is server-owned (src/config/grokConfig.js); client model strings are ignored.
const safeReasoningEffort = VALID_REASONING EffORTS.has(reasoningEffortRaw)
  ? reasoningEffortRaw
  : 'none';
```

```
357:365:api/ai/grok.js
if (debugFullPost) {
  logInfo('grok.debug_fullpost_request', {
    hasModel: Boolean(rawBody.model),
    clientModelRaw: rawBody.model,    // logged for debugging only
    ...
```

```
462:464:api/ai/grok.js
logInfo('grok.upstream_call', { models: [GROK_MODEL], messageCount: normalizedMessages.length });
const response = await runUpstreamOnce(GROK_MODEL);    // server constant, not rawBody.model
```

There is no allowlist because there is nothing to allowlist — the client-supplied value is dead input. This is a correction, not a gap: no fix is needed.

A3

Max tokens per feature

FEATURE / CALL SITE	FILE : LINE	MAX_TOKENS	FEATUREKEY
Hook Builder (standalone)	grokAPI.js:1612,1622	none set	hookBuilder
Full Post — hooks (Grok fallback)	grokAPI.js:1741-1746	1024	fullPostBuilder
Caption (standalone)	grokAPI.js:1042,1052	4096	caption
Full Post — caption	grokAPI.js:1034	8192	fullPostBuilder
Full Post — caption_enhance (Grok fallback)	grokAPI.js:1151-1152	4096	fullPostBuilder
CTA Suggester (standalone)	grokAPI.js:1943 (non-FPB)	4096	cta
Full Post — cta	grokAPI.js:1943 (FPB)	4096	fullPostBuilder
Hashtag Generator (standalone)	grokAPI.js:2256 (non-FPB)	4096	hashtag
Full Post — hashtags (Grok fallback)	grokAPI.js:2256 (FPB)	4096	fullPostBuilder
Quality Scorer (standalone)	grokAPI.js:1248-1251 (non-FPB)	4096	contentQualityScorer

FEATURE / CALL SITE	FILE : LINE	MAX_TOKENS	FEATUREKEY
Full Post — quality_score (Grok fallback)	grokAPI.js:1248-1251 (FPB)	4096	fullPostBuilder
Humanizer (standalone & Full Post, Grok fallback)	grokAPI.js:3305	2048	humanizerScore (never upgraded to fullPostBuilder — see line 3304)
Ignite Engine	igniteEngine.jsx:1464	none set	igniteEngine
Optimize Posting Times	optimizeTimesAPI.js:210	none set	optimizeTimes
Dashboard widgets (3 sites)	dashboardCacheService.js:744,1503,1781	none set	dashboardWidget
Content Repurposer	ContentRepurposer.jsx:131	none set	contentRepurposer

Server ceiling regardless of client value: `api/ai/grok.js:441` caps at `max_tokens ≤ 8192` before forwarding as xAI's `max_completion_tokens`; client-side `grokAPI.js:545` independently caps at `Math.min(n, 8192)`. The four features with no explicit `max_tokens` (Ignite Engine, Optimize Times, dashboardWidget, Content Repurposer) send no field at all — whatever xAI's server-side default is for `grok-4.3` applies, and that default is not defined anywhere in this repo.

A5

Full Post Builder cost & routing

The authoritative source is a comment in the component itself, and it **contradicts** the premise that all 7 steps are pure sequential Grok calls:

```
58:58:src/pages/FullPostBuilder.jsx
* Unchanged routing summary: Hooks Claude→Grok→Hook Builder; Caption Grok; Enhance Claude→Grok;
* Hashtags Perplexity sonar→quick_scan→Grok; CTA Grok; Quality Claude→Grok; Human Claude→Grok;
* Algorithm local checklist (AlgorithmChecker).
```

STEP	PRIMARY	GROK FALLBACK (FEATUREKEY / MAX_TOKENS)	CALLS PERPLEXITY?
hooks	Claude — <code>generateFullPostHooksWithClaude</code> (FullPostBuilder.jsx:655)	fullPostBuilder / 1024 (grokAPI.js:1649-1759)	No
caption	Grok only — <code>generateCaption</code> (FullPostBuilder.jsx:767)	fullPostBuilder / 8192 (grokAPI.js:1034,1038)	No
caption_enhance	Claude — <code>enhanceCaptionWithClaude</code> (FullPostBuilder.jsx:844)	fullPostBuilder / 4096 (grokAPI.js:1112-1157)	No
hashtags	Perplexity — <code>generateFullPostHashtagsGrounded</code> (FullPostBuilder.jsx:918; Sonar full_post_hashtags → quick_scan)	fullPostBuilder / 4096, only if Perplexity insufficient (FullPostBuilder.jsx:939-955)	Yes — primary , not fallback

STEP	PRIMARY	GROK FALLBACK (FEATUREKEY / MAX_TOKENS)	CALLS PERPLEXITY?
cta	Grok only — generateStyledCTAs (FullPostBuilder.jsx:1019, fullPostBuilder/4096)	n/a	No (Perplexity CTA research explicitly skipped when fullPostBuilder=true, grokAPI.js:1843)
quality_score	Claude — scoreContentQuality calls callClaudeAPI first (grokAPI.js:1259)	fullPostBuilder / 4096	No
humanizer	Claude — scoreHumanness (grokAPI.js:3312-3327)	humanizerScore / 2048 (effort stays 'none', deliberately not upgraded — grokAPI.js:3304)	No
(extra, not in the "7")	Algorithm Checker — local rules-based checklist	n/a — no AI call	No

Of the 7 named steps, only 3 (caption, cta, and whichever fallback path fires) are Grok in the *primary* path; 4 have Claude or Perplexity as primary with Grok only firing on failure.

Credits per run: FULL_POST_BUILDER_CREDITS_PER_RUN = FEATURE_CREDIT_COSTS.fullPostBuilderRuns = 4 (creditConfig.js:56, re-exported supabase.js:154). Charged **once per session** at the hooks step (FullPostBuilder.jsx:621) — confirmed by the in-app copy: *"Each run uses 4 credits when hooks generate; caption, hashtags, and CTA steps in the same session do not charge extra runs"* (FullPostBuilder.jsx:1446). Monthly run cap: fullPostBuilderRuns: { essentials: 15, pro: 40, founder: 40, builder: 40 } (creditConfig.js:77).

Section B — Subscription & billing

B1

Writers of public.subscriptions

All write paths are in-repo Vercel functions — no n8n-only writer found.

FILE : LINE	OP	TRIGGER	KEY COLUMNS
stripe-webhook.js:331-333 (updateSubscriptionRecord)	upsert onConflict user_id	checkout.session.completed; customer.subscription.created/updated	tier, status, current_period_start/end, trial_start, trial_end, cancel_at_period_end, cancelled_at, customer_name
stripe-webhook.js:853-861	update	customer.subscription.updated (targeted sync)	status, current_period_start/end
stripe-webhook.js:910-918	update	customer.subscription.deleted	tier:'free', status, cancel_at_period_end:false, cancelled_at
stripe-webhook.js:1041-1047	update	invoice.payment_failed	status:'past_due'
stripe-webhook.js:1129-1132	update	invoice.paid	status:'active', current_period_start/end, cancel_at_period_end:false
stripe-webhook.js:1163-1166	update	invoice.payment_succeeded	status:'active', current_period_start/end
cancel-subscription.js:138-145	update	user-initiated cancellation	cancel_at_period_end, status
change-subscription-plan.js:185-195	update	user-initiated plan change	tier, status, current_period_start/end, cancel_at_period_end, cancelled_at:null
_utils/billing.js:152-155 (syncStripeCustomerId)	update	called from checkout/portal flows	stripe_customer_id

This contradicts the assumption that live traffic only reaches n8n endpoints + a Supabase Edge Function writing the stripe schema. Code cannot confirm which endpoint is actually registered in the live Stripe dashboard — but the repo unambiguously contains a fully built webhook handler that writes public.subscriptions directly.

B2

Stripe webhook handler

Found: `api/stripe-webhook.js`, a standard Vercel serverless function (`bodyParser: false` at lines 88-92 for raw-body signature verification).

- **Signature verification:** yes — `stripe.webhooks.constructEvent(rawBody, sig, endpointSecret)` at line 394, 400 on failure.
- **Registered events:** `checkout.session.completed`, `customer.subscription.trial_will_end`, `customer.subscription.created`, `customer.subscription.updated`, `customer.subscription.deleted`, `invoice.payment_failed`, `invoice.paid`, `invoice.payment_succeeded` (switch at line 407).
- **Idempotency:** a `stripe_webhook_events` table (lines 218-258).

B3

Checkout Session creation

```
161:173:api/create-checkout-session.js
const subscriptionData = {
  metadata: baseMetadata,
  ...(!!isLaunchPricingPlan && !isAnnualBilling
    ? {
      trial_period_days: 7,
      trial_settings: {
        end_behavior: {
          missing_payment_method: 'cancel',
        },
      },
    }
    : {}),
};
```

- `trial_period_days: 7` — only for non-launch (Essentials/Pro), non-annual (monthly) plans.
- `payment_method_collection: 'always'` (line 182).
- `trial_settings.end_behavior.missing_payment_method: 'cancel'` (line 168).

Matches the note that all four live prices have `trial_period_days: null` at the Price object level — the 7-day trial is entirely session-level.

B4

`/api/trial-reminder-cron`

- **Cron schedule** (vercel.json:13-16): `"0 13 * * *"` — daily at 13:00 UTC.
- **Columns queried:** `subscriptions.stripe_subscription_id` filtered by `status='trialing'`, `trial_end` not null, within a 73h window (lines 39-45) — it does query `trial_end`, consistent with the cited production failure.
- **Emails triggered:** via `maybeSendTrialReminder` (`trialReminderUtils.js:107-152`) → `sendTrialWarningEmail`, gated to 3-day or 1-day remaining.

Adjacent findings (not asked, but material):

- The migration that creates `trial_end` already exists in the repo — `supabase/migrations/20260311180000_add_trial_reminders_and_subscription_trial_fields.sql:1-3`, dated March 2026. If production still lacks the column, this is migration drift (never actually applied to prod), not a missing migration.
- Separate bug: the DB constraint only allows `reminder_type IN ('trial_2_days','trial_1_day')` (same migration, line 13), but `trialReminderUtils.js:118` writes `'trial_3_days'` for the 3-day case — a value the CHECK constraint would reject.

B5

Hardcoded Stripe price IDs

No price IDs are hardcoded in application source — all four live prices come from env vars (`stripeAPI.js:145-197`, `create-checkout-session.js:144-149`, `_utils/stripePlans.js:35-46`).

Flagged — a 5th price ID not in the four-price list, found in the local (git-ignored) `.env` :

```
.env:52: VITE_STRIPE_PRICE_BUILDER_ANNUAL=price_1TG2ZjAlegFQU8JhYu8V9N2r
```

Consumed at `stripeAPI.js:177` as the “Legacy Annual” / Builder tier price, referenced in `_utils/stripePlans.js:45` and present in the built client bundle. Confirmed not git-tracked (`git ls-files .env` returns nothing). `.env.example` only lists placeholder `price_xxx_*` values and doesn’t even declare a `BUILDER_ANNUAL` var name. No n8n config JSON in the repo contains price IDs.

Section C — Tier limits

C1 · HIGHEST PRIORITY

14-day Plan Builder gating

A) GATED to Pro at `api/create-plan-builder-job.js:106` (duplicated at `api/plan-builder-proxy.js:420`).

```
103:116:api/create-plan-builder-job.js
// 3a. 14-day tier gate — Essentials cannot request 14-day plans.
//     Matches creditConfig.PLAN_BUILDER_14DAY_TIERS on the client.
const isFourteenDay = Number(period) === 14;
if (isFourteenDay && !PLAN_BUILDER_14DAY_ALLOWED_TIERS.includes(userTier)) {
  ...
  return res.status(403).json({
    error: 'tier_restricted',
    message: '14-day plans require Pro or above.',
  });
}
```

`PLAN_BUILDER_14DAY_ALLOWED_TIERS = ['pro', 'founder', 'builder']` (`planBuilderLimits.js:27`), mirrored client-side by `PLAN_BUILDER_14DAY_TIERS` (`creditConfig.js:111`). Belt-and-suspenders:

`PLAN_BUILDER_14DAY_MONTHLY_BY_TIER.essentials = 0` (`planBuilderLimits.js:20`), so even a bypassed tier check still resolves `cap: 0` and 403s.

This is a hardcoded array/tier check — **not** derived from the `tier_feature_limits` table. A repo-wide search for `tier_feature_limits` returns zero matches in `src/`, `api/`, or `supabase/migrations/`.

C2

Credit-checking module

Module: `src/hooks/useAIUsage.js` + `src/config/creditConfig.js`, mirrored server-side in `api/create-plan-builder-job.js` / `api/plan-builder-proxy.js`.

- **Run caps or total_credits only?** Both, as a two-step gate (`useAIUsage.js:24-31`): Step A is a per-feature monthly run cap from `FEATURE_RUN_CAPS[featureName][userTier]` (`creditConfig.js:62-83`), checked first; Step B is the shared credit pool.
- **Tier key during a trial:** `userTier` comes straight from `subscriptions.tier` (plain value: `essentials/pro/founder/builder`). Trial state is a *separate* boolean, `isTrialing = subscription?.status === 'trialing'` (`SubscriptionContext.jsx:583`) — the tier string is never mutated for trial state.
- **Any path writing a `_trial`-suffixed tier? No.** `resolvePlanId()` (`stripePlans.js:79`, map at 9-21) only resolves to `essentials/pro/builder/founder`. A repo-wide search for `_trial` as a tier literal returns zero matches in any `.js` / `.jsx` file.

Conclusion: the 12 per-feature caps added for `essentials_trial / pro_trial` directly in the `tier_feature_limits` table on 2026-08-25 have **zero effect** on enforcement — no application code reads that table, and no code path ever produces a tier value of `essentials_trial / pro_trial` to look up in the first place. `FEATURE_RUN_CAPS` in `creditConfig.js` is the real, hardcoded, and only source of per-feature caps.

Section D — Dashboard & auth

D1

daily_dashboard_cache read query

```
2017:2025:src/services/dashboardCacheService.js
const fullSelect =
  'generated_date, trending_topics, hashtags_of_day, ai_insight, ai_insights, daily_alerts,
dashboard_metadata, created_at';

const { data, error } = await supabase
  .from(DASHBOARD_CACHE_TABLE)
  .select(fullSelect)
  .eq('user_id', userId)
  .eq('generated_date', dateKey)
  .maybeSingle();
```

This row read is **per-user**. The *trending widget cache* (a separate table, `niche_content_cache`) is **per-niche**: its key is built with no `user_id` component (`buildPerPlatformCacheKey`, lines 490-498), and the code comment states directly: *“Perplexity proxy writes with user_id null for dashboard trending/hashtags so all accounts in the same niche reuse one payload.”*

Strip-and-retry block (`dashboardCacheService.js:1932-1959`):

```
1948:1958:src/services/dashboardCacheService.js
const badColumn = isMissingColumn ? parseMissingColumnName(error) : null;
if (!badColumn || !(badColumn in currentRow) || badColumn === 'user_id' || badColumn ===
'generated_date') {
  return { error };
}
console.warn(`[Dashboard] daily_dashboard_cache upsert: stripping unknown column "${badColumn}"
and retrying.`);
const { [badColumn]: _removed, ...rest } = currentRow;
currentRow = rest;
}
return { error: new Error('daily_dashboard_cache upsert failed after stripping unknown columns.') };
```

The read side has its own three-tier fallback: `fullSelect` (includes `dashboard_metadata`) → legacy select (drops it) → minimal select (drops it and `ai_insights`), lines 2010-2056.

Conflict for Section E: the repo’s own most recent migration, `20260719014640_reconcile_production_cache_schema_baseline.sql:27-28`, explicitly **drops** `dashboard_metadata` because it was “confirmed absent from production.” The live code above still actively selects it as first choice, with fallbacks built to survive its absence — an internal gap between migration history and defensive code, predating the Aug 25 changes.

D2

`/api/ai/perplexity` **timeout / concurrency / retry**

- **maxDuration:** 60 seconds (vercel.json:34-36).
- **Upstream call pattern:** exactly one `fetch(PERPLEXITY_API_URL, ...)` per invocation (perplexity.js:783-788) — no internal loop.
- **Server-side retry/backoff: NOT FOUND** — no retry loop around the upstream fetch inside the route itself.
- **Client-side retry/backoff:** yes — `src/utils/retryFetch.js:21`, configured via `apiConfig.js:8-12` (MAX_RETRIES:2, BASE_DELAY_MS:750, retryable on 408/425/429/500/502/503/504, exponential backoff + jitter). This wraps some client call sites but never runs inside the serverless function, and a 504 from hitting the 60s wall would itself be retried — compounding rather than fixing a 429 problem, since each retry consumes another slot against the route's own 20/min rate limit (perplexity.js:148).

D3

`supabase.auth.signOut()` call sites

```
566:588:src/context/AuthContext.jsx
const logout = async () => {
  try {
    // Local scope clears persisted session first so refresh-token work stops immediately
    await supabase.auth.signOut({ scope: 'local' });
    try {
      await supabase.auth.signOut({ scope: 'global' });
    }
    ...
    return { success: true };
  } catch (error) {
    console.error('Logout error:', error);
    try {
      await supabase.auth.signOut({ scope: 'local' });
    }
  }
}
```

Only call sites in the repo, both inside the single `logout()` function in `AuthContext.jsx`. This is only invoked by explicit user action — no call to `logout()` was found gated on subscription status, cancellation, or payment failure. Cancellation/payment-failure flows (`stripe-webhook.js`, `cancel-subscription.js`) only touch the `subscriptions` table and never call sign-out.

No forced sign-out on billing events found.

Section E — Migration state

E1

Ten most recent migrations (by actual git commit date)

Three legacy files without timestamp prefixes sort last alphabetically but are actually the *oldest* (January 2026) — ranked here by true git history, not filename sort.

DATE	FILE
2026-07-18	20260719014640_reconcile_production_cache_schema_baseline.sql
2026-04-25	20260425120000_expand_subscription_status_constraint.sql
2026-03-31	20260331120000_add_audience_stage_to_user_profile.sql
2026-03-30	20260330130000_add_secure_account_email_sent.sql
2026-03-30	20260330120000_daily_dashboard_cache_update_policy_with_check.sql
2026-03-27	20260327120000_add_post_kits_kit_slots.sql
2026-03-24	20260323120000_add_has_seen_welcome_notification.sql
2026-03-24	20260320120000_add_missing_user_profile_columns.sql
2026-03-20	20260320150000_user_preferences_rls_fix.sql
2026-03-19	20260319200000_user_profile_upsert_rls_with_check.sql

The newest migration in the repo predates 2026-08-25 by over five weeks — there is nothing in the repo from the date the production changes were made.

E2

Migration references vs. the Aug 25 production changes

- `subscriptions.trial_end` : found, but only in the older `20260311180000_add_trial_reminders_and_subscription_trial_fields.sql:1-3` (adds the column). No migration drops it. Unrelated to the Aug 25 change; if this migration was genuinely never run in prod, re-running it would *fix*, not duplicate, the gap.
- `daily_dashboard_cache.dashboard_metadata` : found — and it’s the opposite of helpful. `20260719014640_reconcile_production_cache_schema_baseline.sql:27-28` explicitly **drops** this column, asserting it’s “confirmed absent from production.” If it was added back to prod directly on 2026-08-25, a future re-run of this migration would **actively conflict** with that out-of-band change.
- `tier_feature_limits rows for essentials_trial / pro_trial` : **NOT FOUND**. No migration file references this table at all — it isn’t created, altered, or seeded anywhere in `supabase/migrations/`. It exists only in production, entirely outside repo history. A future migration would need a from-scratch `CREATE TABLE IF NOT EXISTS` with no baseline to build on.

Summary of corrections to the audit brief

1. **Grok model is not client-controlled.** `clientModelRaw` is logged for debugging but the upstream call always uses the server-owned `GROK_MODEL` constant. No allowlist exists because there's nothing for it to gate.
2. **grok-4.3-migration is already merged** (PR #96) — `main` is strictly ahead, not a separate unmerged branch.
3. **Full Post Builder is not "7 sequential Grok steps."** 4 of the 7 named steps have Claude or Perplexity as the primary call, with Grok only as a fallback on failure.
4. **14-day Plan Builder access is Pro-gated in code** (`create-plan-builder-job.js:106`), independent of and not derived from `tier_feature_limits`.
5. **tier_feature_limits is read by zero application code and zero migrations.** The Aug 25 `essentials_trial / pro_trial` rows have no functional effect; `FEATURE_RUN_CAPS` in `creditConfig.js` is the real, hardcoded enforcement source, and it doesn't recognize `_trial`-suffixed tiers (which no code path produces anyway).
6. **subscriptions.trial_end migration already exists in the repo** (March 2026) — the production error suggests it was never applied there, not that a migration is missing.
7. A repo migration (`20260719014640`) already **removed** `dashboard_metadata` on the assumption production lacked it — now stale/contradicted if production has that column again.

End of report. Generated from a static, read-only review of `main` @ `a1f3674`. No files were changed, no migrations were run, no branches were checked out, and no destructive or write commands were issued during this audit.